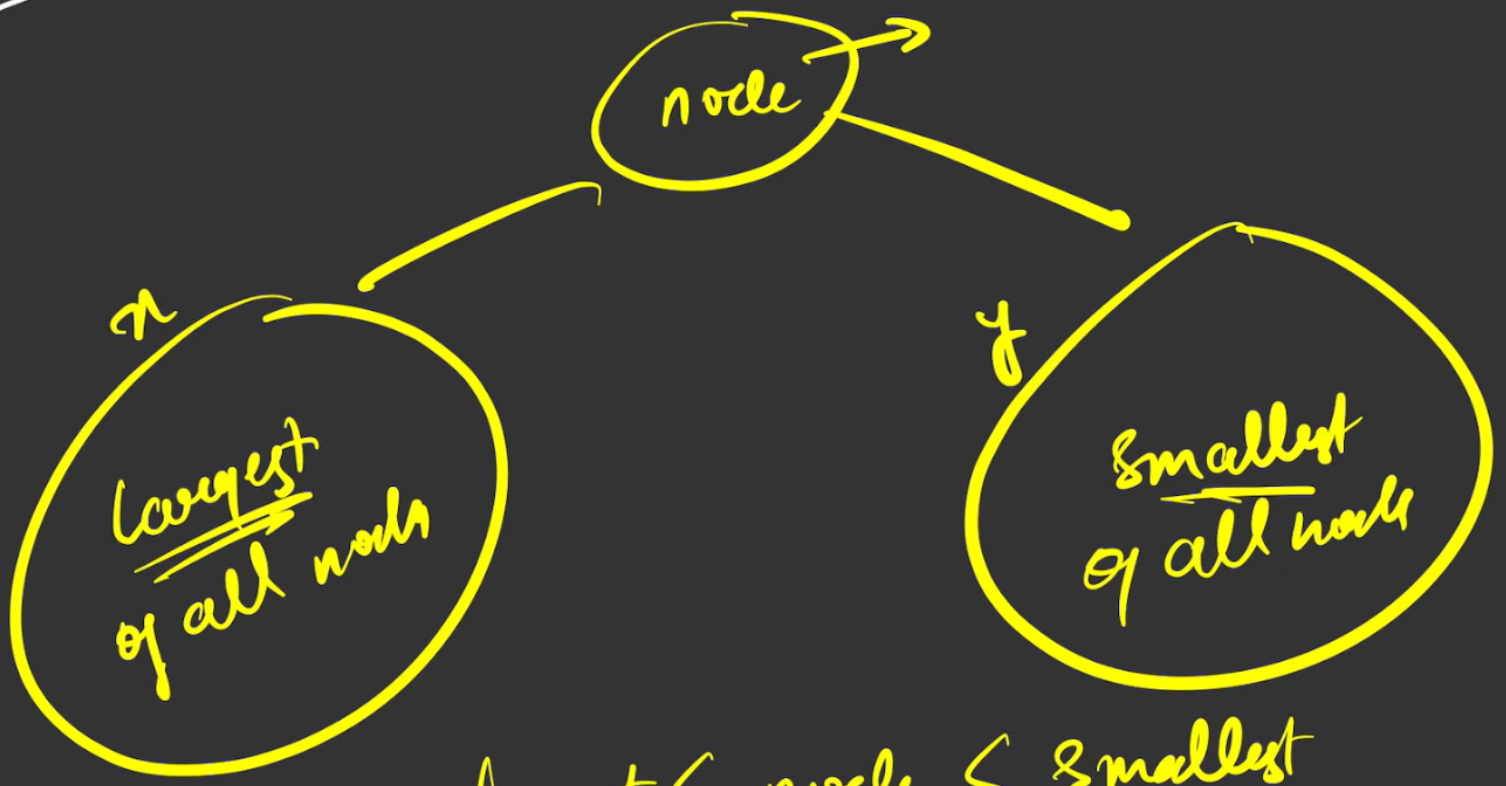


Intuition | Thought Process

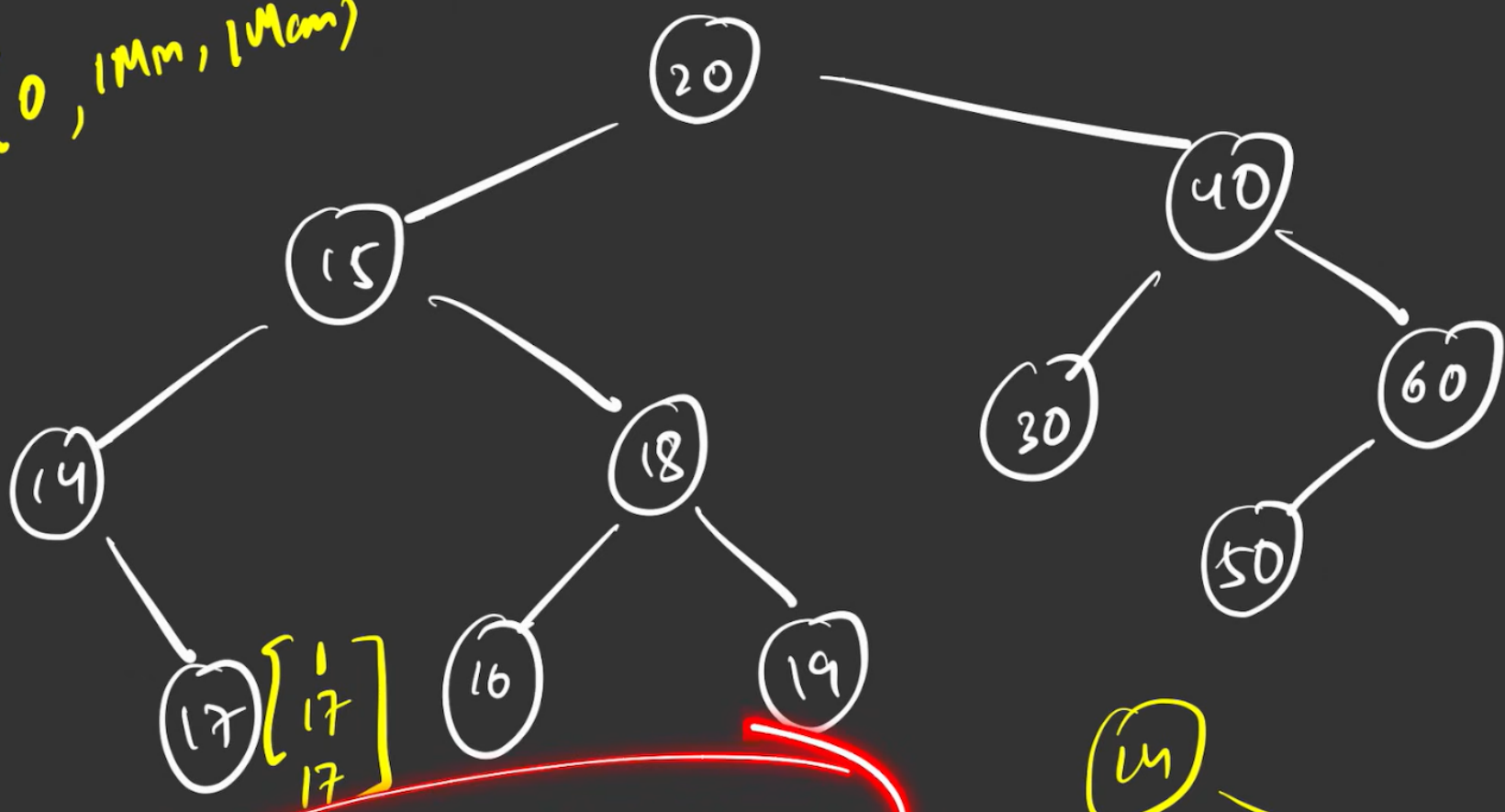
size $\rightarrow 1 + n + y$



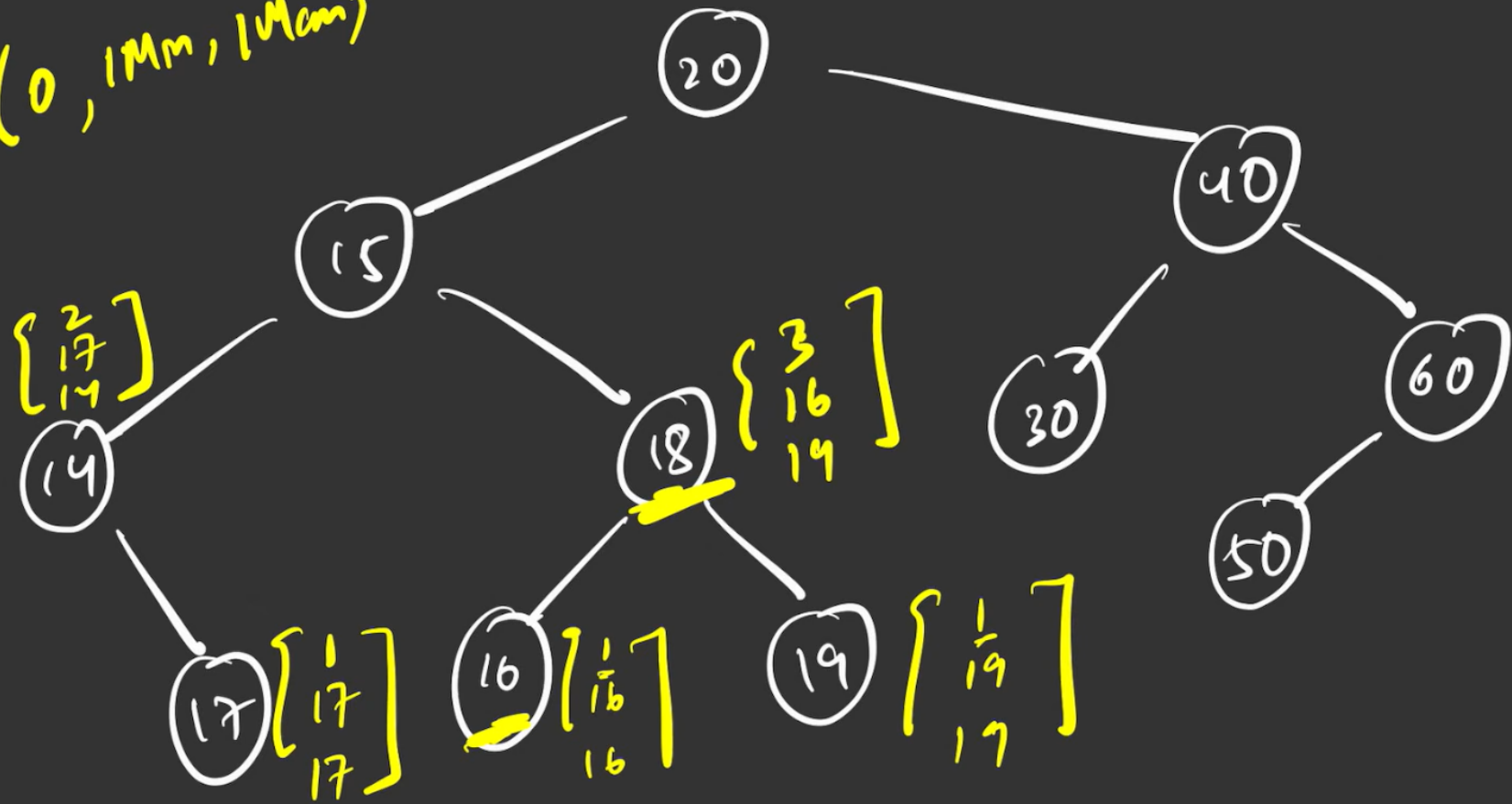
$\text{Largest} < \underline{\text{node}} < \text{Smallest}$



(0, 1Mm, 1Mm)

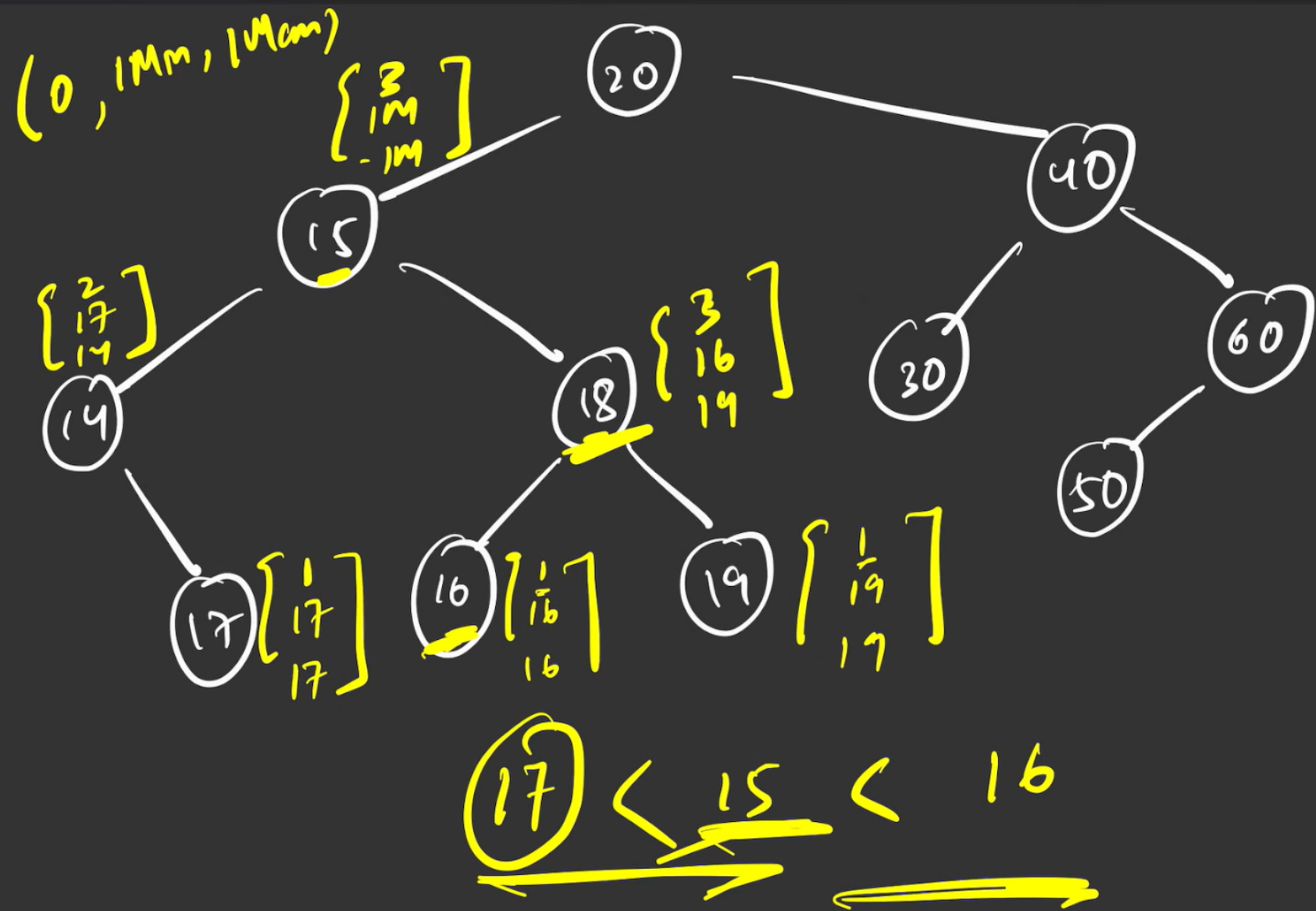


(0, 1Mm, 1Mm)



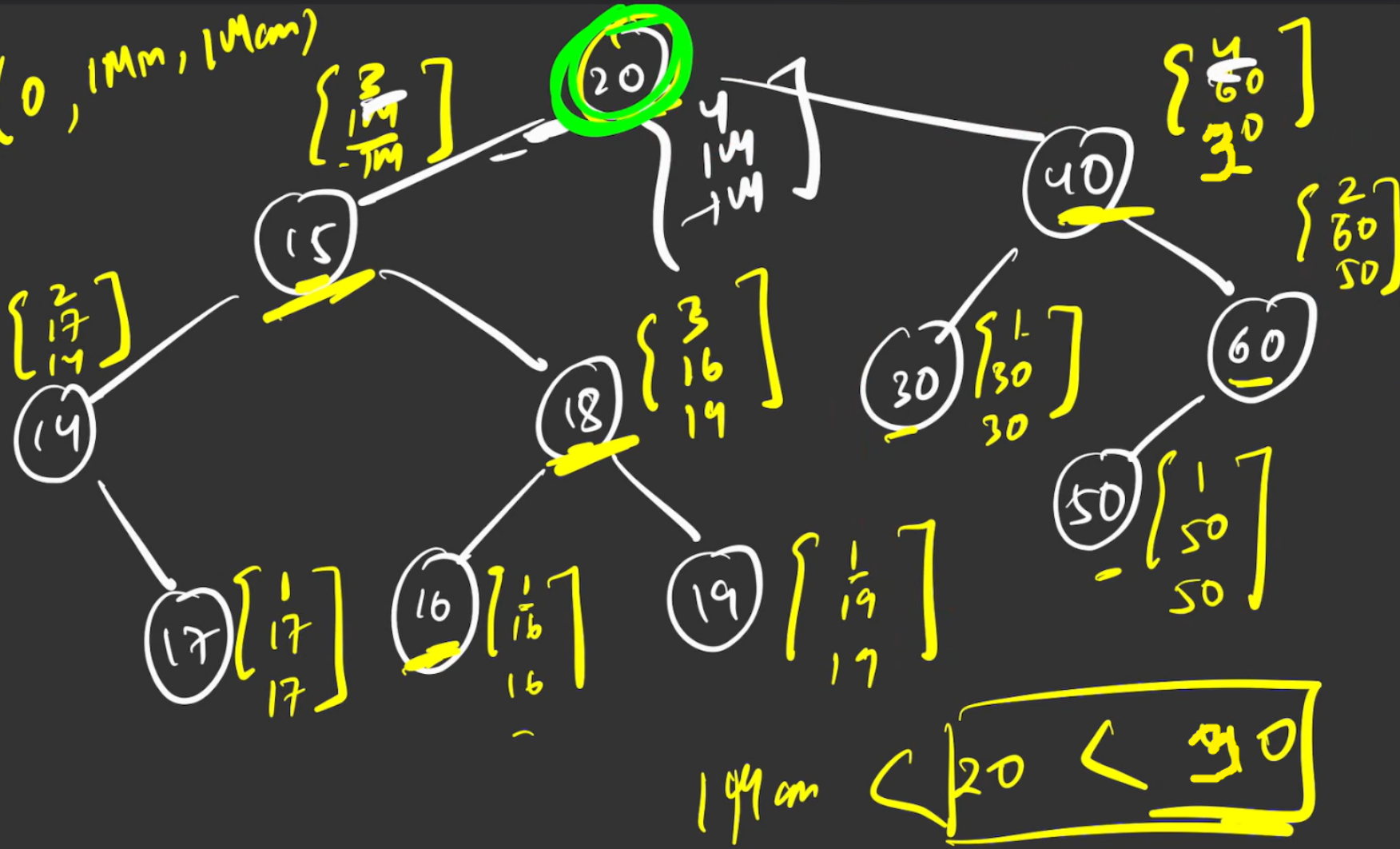
$16 < 18 < 19$





} node }

(0, 1Mm, 1Mm)



🔥 Two Different Concepts

1. ✅ For validation:

`// C++`

```
left.max < root < right.min
```



👉 uses:

- max from left
- min from right

2. ✅ For returning values (IMPORTANT)

After confirming it's BST, we return:

`// C++`

```
min = minimum of whole subtree  
max = maximum of whole subtree
```



🔍 Apply to Node 40

Tree:

`// C++`

```
    40  
   /  \  
  30   60  
   /  
  50
```



✅ Step 1: Validate BST

- left max = 30
- right min = 50

`// C++`

```
30 < 40 < 50 ✅
```



✅ Step 2: Return values

Now we compute:

`// C++`

```
min = min(40, left.min) = min(40, 30) = 30  
max = max(40, right.max) = max(40, 60) = 60
```



(left, right, size)

If not BST

size → mem (left-size, right-size)

MAX → 1000000

Mn → 1000000

postorder

left
right



L53. Largest BST in Binary Tree

1.60

Autocomplete

i {}

```
* TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
* TreeNode(int x, TreeNode *left, TreeNode *right) : val(x), left(left), right(right) {}
* };
*/
class NodeValue {
public:
    int maxNode, minNode, maxSize;

    NodeValue(int minNode, int maxNode, int maxSize) {
        this->maxNode = maxNode;
        this->minNode = minNode;
        this->maxSize = maxSize;
    }
};

class Solution {
private:
    NodeValue largestBSTSubtreeHelper(TreeNode* root) {
        // An empty tree is a BST of size 0.
        if (!root) {
            return NodeValue(INT_MAX, INT_MIN, 0);
        }

        // Get values from left and right subtree of current tree.
        auto left = largestBSTSubtreeHelper(root->left);
        auto right = largestBSTSubtreeHelper(root->right);

        // Current node is greater than max in left AND smaller than min in right, it is a BST
        if (left.maxNode < root->val && root->val < right.minNode) {
            // It is a BST.
            return NodeValue(min(root->val, left.minNode), max(root->val, right.maxNode),
                             left.maxSize + right.maxSize + 1);
        }

        // Otherwise, return [-inf, inf] so that parent can't be valid BST
        return NodeValue(INT_MIN, INT_MAX, max(left.maxSize, right.maxSize));
    }
public:
    int largestBSTSubtree(TreeNode* root) {
        return largestBSTSubtreeHelper(root).maxSize;
    }
};
```

previous code was restored from your local storage. [Reset to default](#)

sole - Contribute i

▶ Run Code ^

16:48 / 17:26

Code >